

PLATAFORMA ADMINISTRATIVA PARA LA PARROQUIA SAN PEDRO NOLASCO**PLAN MAESTRO DE PRUEBAS**

Diego Andre Calderón Salazar - 241263

Pedro Julio Caso - 241286

Javier Sebastián Alvarado Monzón - 24546

Hugo Méndez Lee - 241265

José Miguel Rosas Guerra - 241274

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3091 - Ingeniería de Software 2 - Sección 30

Semestre II - 2026

Plan maestro de pruebas

Preparado por: Diego Andre Calderón Salazar, Pedro Julio Caso, Javier Sebastián Alvarado

Monzón, Hugo Méndez Lee y José Miguel Rosas Guerra.

Introducción

El presente Plan Maestro de Pruebas define la estrategia, alcance y criterios de calidad aplicados al desarrollo de la Plataforma Administrativa para la Parroquia San Pedro Nolasco durante este sprint. Las actividades de prueba se centran en la verificación funcional y unitaria de los componentes visuales y mecanismos internos del sistema (frontend y backend), priorizando los flujos clave de autenticación, gestión de reservas, asignación de tareas y el sistema de notificaciones de inasistencia, así como las pruebas de regresión para asegurar la estabilidad integral. Quedan formalmente excluidas del alcance de este sprint las evaluaciones especializadas de carga, estrés y pruebas severas de seguridad.

La ejecución de las pruebas se distribuye de manera equitativa entre los cinco miembros del equipo de desarrollo (con un 20% de participación cada uno) y se rige bajo criterios de aceptación normados en la integración continua mediante GitHub Actions, exigiendo cobertura en controladores críticos y el bloqueo de fusiones ante fallos. Asimismo, se establecen mecanismos de suspensión y reanudación antes defectos bloqueantes, indisponibilidad del entorno o ambigüedades en requisitos. Finalmente, el plan contempla la gestión de riesgo operativos y técnicos significativos, tales como la desalineación en las historias de usuario, problemas en volúmenes de Docker, regresiones por integración, falsos negativos por tiempo de ejecución y la idoneidad de las pruebas de usabilidad dirigidas a la población objetivo de la parroquia.

Recursos

Tester	% de participación
Javier Alvarado	20
Diego Calderón	20
Pedro Caso	20
Hugo Méndez	20
Miguel Rosas	20

Alcance

Corresponde a presentar y delimitar la totalidad del trabajo que es necesario para dar por terminado el plan de pruebas.

El alcance de este plan de pruebas incluye la validación de los componentes nuevos desarrollados y la validación integral del sistema, asegurando el funcionamiento correcto desde la UI hasta persistencia en la base de datos. Esto incluye de manera general las pruebas funcionales, pruebas de integración mediante frameworks, pruebas de regresión aplicadas a características específicas del sistema, pruebas de compatibilidad y pruebas de sistema si entra dentro del límite de tiempo de entrega del proyecto. El objetivo principal es asegurar que todos los módulos del sistema interactúen de forma correcta y esperada de acuerdo con el diseño y arquitectura definidos con antelación en el proyecto.

Para el presente sprint, las actividades de prueba se limitaron a pruebas unitarias de mecanismos internos del sistema tanto de backend como frontend, enfocándose en las historias de usuario de alta prioridad, dentro de esta fase, se tiene como objetivo probar componentes visuales y funcionamiento interno del sistema.

Fuera del alcance

Se deben incluir los elementos del sistema que no se probarán como parte del plan.

Los elementos del sistema que no serán incluidos o bien, las pruebas que quedan excluidas en este plan incluyen las pruebas sobre especializadas en carga, estrés y seguridad. Por lo que, no se evaluará el comportamiento o estabilidad del sistema bajo condiciones extremas de tráfico de usuarios ni análisis de vulnerabilidades severo para la protección del sistema durante el plan para darlo como completo.

Características a probar

- Descripción de las pruebas a realizar.
 - Se establecerá el rango de las fechas en las que el sistema será probado.
 - Se describirán las pruebas que se realizarán al sistema utilizando las plantillas siguientes:

Pruebas de funcionalidad

ID caso de prueba	Escenario	Variable 1 (Rol / Usuario)	Variable 2 (Datos de entrada / condición)	Resultado esperado
1.1	Login con credenciales válidas	Cualquier rol	Precondición: persona registrada en seeds. Entrada: correo + password correctos	Token JWT generado, redirige a /dashboard, usuario guardado en localStorage
1.2	Login con contraseña incorrecta	Cualquier rol	Precondición: persona registrada. Entrada: correo válido + password incorrecto	401, sin token generado, mensaje de error visible
1.3	Acceso a ruta protegida sin sesión	—	Precondición: sin token en localStorage. Entrada: navegar a /dashboard directo	Redirige a /login
1.4	Renovación automática de sesión (HU-24)	Cualquier rol	Precondición: access token expirado (15 min), refresh vigente. Entrada: petición API tras expiración	Interceptor llama POST /api/auth/refresh, reintenta petición sin interrumpir al usuario
2.1	Refresh token expirado (HU-26)	Cualquier rol	Precondición: refresh token vencido (>15 días). Entrada: petición API con refresh vencido	Notificación de sesión expirada, redirige a /login
2.2	Ruta inexistente (HU-25)	Cualquier rol autenticado	Entrada: navegar a URL no definida	Se muestra NotFoundPage (404)
3.1	Registro de persona	Sacerdote/Admin	Entrada: datos válidos de nueva persona	POST /api/auth/register responde 201, persona

	autorizado (HU-28)			creada con rol asignado
3.2	Registro de persona no autorizado	Ministro/Coordinador	Entrada: datos válidos de nueva persona	403 Forbidden, no se crea el registro
3.3	Registro con correo duplicado	Sacerdote/Admín	Precondición: correo ya existe en persona. Entrada: correo duplicado	Error de validación, no se crea duplicado
4.1	Crear reserva con evento (HU-13)	Cualquier rol autenticado	Precondición: espacio disponible. Entrada: espacio_id, fecha, hora_inicio, hora_fin, título, descripción	POST /api/reservas crea reserva (estado=Pendiente) + evento en transacción atómica
4.2	Reserva con horario en conflicto (HU-16)	Cualquier rol	Precondición: reserva Confirmada existente en ese espacio/horario. Entrada: mismo espacio_id, horario solapado	Sistema bloquea la creación/aprobación, responde con error de conflicto
4.3	Reserva con horas inválidas	Cualquier rol	Entrada: hora_inicio >= hora_fin	400 Bad Request
5.1	Aprobar reserva pendiente (HU-02)	Sacerdote/Admín	Precondición: reserva Pendiente sin conflicto. Entrada: click "Aprobar"	estado_reserva_id cambia a Confirmada (2)
5.2	Rechazar reserva pendiente (HU-02)	Sacerdote/Admín	Precondición: reserva Pendiente. Entrada: click "Rechazar"	estado_reserva_id cambia a Rechazada (3), espacio queda libre
6.1	Cancelar reserva propia (funcionalidad nueva, D3)	Solicitante	Precondición: reserva Pendiente/Confirmada propia. Entrada: click "Cancelar" + confirmación	Estado cambia a Rechazada/Cancelada, espacio liberado automáticamente (reutiliza lógica HU-27)
6.2	Cancelar reserva ajena sin permisos	Coordinador (no dueño, no Admin/Sacerdote)	Precondición: reserva de otro solicitante. Entrada: intento de cancelar	403 Forbidden
6.3	Admin/Sacerdote cancela reserva ajena	Admin/Sacerdote	Precondición: reserva de cualquier solicitante. Entrada: click "Cancelar"	Permitido, estado actualizado y espacio liberado
7.1	Asignar tarea sin conflicto (HU-06)	Coordinador de Ministros	Precondición: ministro sin tarea en ese horario. Entrada: tarea_id + persona_id	Asignación creada, notificación automática generada al ministro (HU-05)
7.2	Asignar tarea con conflicto de horario (HU-08)	Coordinador de Ministros	Precondición: ministro ya asignado a otra tarea en el mismo horario. Entrada: tarea_id + persona_id en conflicto	409 Conflict, asignación no se crea
7.3	Desasignar tarea	Coordinador de Ministros	Precondición: ministro con tarea asignada. Entrada: DELETE asignación	Registro de asignación_tarea eliminado
8.1	Notificación de inasistencia con	Ministro	Precondición: tarea/servicio ya asignado. Entrada: click "No	Excepción registrada (fecha, evento, motivo) y

	motivo (HU-10, CA-03)		podré asistir" + motivo obligatorio	notificación individual inmediata al coordinador de ministros correspondiente
8.2	Notificación de inasistencia sin motivo	Ministro	Precondición: tarea asignada. Entrada: click "No podré asistir" sin motivo	Sistema rechaza la solicitud, motivo obligatorio no cumplido
8.3	Inasistencia sobre tarea no asignada al ministro	Ministro	Precondición: tarea asignada a otro ministro. Entrada: intento de marcar inasistencia	403/404, no se registra la excepción
8.4	Coordinador recibe notificación de inasistencia	Coordinador de Ministros	Precondición: excepción registrada (escenario anterior). Entrada: consultar /api/notificaciones	Notificación tipo individual visible, con motivo y datos del servicio
9.1	Disponibilidad de espacios sin reservas (HU-29)	Cualquier rol	Precondición: sin reservas confirmadas en fecha/hora. Entrada: GET /api/espacios?fecha=&hora_inicio=&hora_fin=	Todos los espacios retornan disponible: true
9.2	Disponibilidad de espacio ocupado (HU-29)	Cualquier rol	Precondición: reserva Confirmada existente. Entrada: mismos parámetros de esa reserva	Ese espacio retorna disponible: false
9.3	Consulta con parámetros incompletos	Cualquier rol	Entrada: solo fecha (sin hora_inicio/hora_fin)	400 Bad Request
10.1	Editar perfil propio	Cualquier rol	Entrada: nombre/correo/password nuevos	PUT /api/personas/:id actualiza campos enviados, password rehasheado
10.2	Cambio de rol_id sin ser Admin	Ministro/Coordinador/Sacerdote	Entrada: envía rol_id en el body	403 Forbidden
10.3	Editar perfil de otra persona sin ser Admin	Cualquier rol no-Admin	Entrada: PUT sobre persona_id distinto al propio	403 Forbidden
10.4	Correo duplicado al editar	Cualquier rol	Precondición: correo ya usado por otra persona. Entrada: nuevo correo ya existente	409 Conflict

Pruebas de carga

Identificador de la prueba	Parte de la aplicación probada	Condición	Resultado esperado	Método o herramienta a utilizar
PC-AUTH-01	Módulo de Autenticación (POST /api/auth/login)	Carga nominal esperada de 10 a 15 VUs realizando inicio de sesión simultáneo en horas pico habituales.	Código HTTP 200 OK con token JWT, 0% de errores, tiempo p90 < 800 ms y consumo moderado de CPU.	Grafana k6 (01_auth_login.js) en contenedor Docker.
PC-ESP-01	Módulo de Espacios (GET /api/espacios)	Concurrencia típica de 15 a 25 VUs consultando disponibilidad de salones con filtros de horario.	Código 200 OK, 0% de fallos, latencia promedio < 5 ms (p95 < 50 ms) y uso de < 50% del pool de BD.	Grafana k6 (02_espacios_disponibilidad.js) en Docker.
PC-NOTIF-01	Módulo de Notificaciones (GET /api/notificaciones)	Sondeo periódico regular cada 60s por 30 a 50 usuarios activos	Código 200 OK, 0% errores, respuesta inmediata (p95 < 15 ms) y mínimo impacto en el ancho de banda.	Grafana k6 (03_notificaciones_polling.js) en Docker
PC-RES-01	Módulo de Reservas (POST /api/reservas)	10 a 15 coordinadores creando solicitudes de reserva simultáneas en distintos salones.	Código 201 Created, inserción transaccional íntegra en estado 'Pendiente' y latencia p90 < 500 ms.	Grafana k6 con peticiones autenticadas hacia la API.

Identificador de la prueba	Parte de la aplicación probada	Condición	Resultado esperado	Método o herramienta a utilizar
PC-CAL-01	Módulo de Calendario (GET /api/eventos)	20 a 30 usuarios simultáneos consultando la vista del calendario parroquial mensual.	Código 200 OK con el listado de eventos y relaciones en < 150 ms, sin bloqueos en la base de datos.	Grafana k6 simulando consultas al calendario.
PC-E2E-01	Flujo Integral de Usuario (User Journey)	Simulación de 15 a 25 usuarios ejecutando el ciclo de vida completo (Login, Notificaciones, Espacios).	Tasa de fallos 0.00%, 100% aserciones aprobadas y latencia global p90 < 400 ms.	Grafana k6 (04_scenari_combined.js) en Docker.

Pruebas de estrés

Identificador de la prueba	Parte de la aplicación probada	Condición	Resultado esperado	Método o herramienta a utilizar
PE-AUTH-01	Autenticación (POST /api/auth/login)	Sobrecarga de 35 a 50 VUs continuos ejecutando cómputo intensivo de hashing bcrypt.	Servidor encola solicitudes sin colapsar ni dar 500/504; degradación controlada (p95 ~ 2.9s) y recuperación normal.	Grafana k6 (01_auth_login.js) en Docker.

Identificador de la prueba	Parte de la aplicación probada	Condición	Resultado esperado	Método o herramienta a utilizar
PE-ESP-01	Módulo de Espacios (GET /api/espacios)	Saturación de 60 a 80 VUs superando en 8x la capacidad del pool de conexiones (10 conexiones).	0% errores de conexión, resolución óptima por índices en memoria (p95 < 5 ms) y cola del pool sin desbordar.	Grafana k6 (02_espacios_disponibilidad.js) en Docker.
PE-NOTIF-01	Notificaciones (GET /api/notificaciones)	Avalancha de sondeo masivo de 100 a 120 VUs ejecutando múltiples JOIN en MariaDB.	0% de fallos HTTP, latencia p95 < 10 ms, sin fugas de memoria en Node.js ni bloqueo de MariaDB.	Grafana k6 (03_notificaciones_polling.js) en Docker.
PE-RES-01	Detección de Colisiones (POST /api/reservas)	30 a 40 usuarios intentando reservar el mismo espacio y horario exactamente al mismo tiempo.	Prevención de doble reserva: 1 solicitud aceptada (201) y el resto rechazadas (400) sin deadlocks en BD.	Grafana k6 con envío concurrente de payloads idénticos
PE-SPIKE-01	Resiliencia a Picos (Spike Testing)	Incremento súbito de 0 a 100 VUs en solo 5 segundos contra el backend completo.	Tolerancia a la avalancha sin caída de contenedores, encolamiento temporal y retorno a la normalidad en < 15s.	Grafana k6 con ejecutor ramping-arrival-rate.

Identificador de la prueba	Parte de la aplicación probada	Condición	Resultado esperado	Método o herramienta a utilizar
PE-SOAK-01	Resistencia Continua (Soak Testing)	Carga sostenida de 25 VUs durante 30 a 45 minutos continuos de tráfico ininterrumpido.	Estabilidad a largo plazo: curva de memoria plana en Node.js y 0 fugas de conexiones en el pool de MariaDB.	Grafana k6 ejecutando prueba prolongada de estabilidad.

Pruebas de seguridad

Identificador de la prueba	Condición	Elemento a probar	Resultado esperado
PS-SEG-01	Backend configurado con un origen autorizado	Configuración CORS	La API acepta solicitudes solo desde el origen definido en CORS_ORIGIN.
PS-SEG-02	Solicitud desde un origen no autorizado	Configuración CORS	La API rechaza la solicitud sin exponer detalles internos ni devolver error 500.
PS-SEG-03	Variables JWT presentes en el entorno	Secretos JWT	JWT_SECRET y JWT_REFRESH_SECRET se obtienen únicamente de variables de entorno; no existen secretos de respaldo hardcodeados.
PS-SEG-04	Usuario con rol Ministro autenticado	Ruta administrativa de eventos	Al intentar crear, editar o eliminar un evento, el servidor responde 403 Forbidden.
PS-SEG-05	Usuario con rol Ministro autenticado	Rutas de gestión de tareas	El usuario no puede crear, editar, eliminar ni asignar tareas si su rol no cuenta con permiso.

Identificador de la prueba	Condición	Elemento a probar	Resultado esperado
PS-SEG-06	Usuario con rol Ministro autenticado	Rutas de gestión de grupos	El usuario no puede crear, editar ni eliminar grupos sin el rol autorizado.
PS-SEG-07	Credenciales inválidas o texto de inyección	Inicio de sesión	La API responde 401 y no autentica al usuario.
PS-SEG-08	Texto ' OR '1'=1 enviado en campos de texto	Consultas SQL de autenticación, tareas, eventos e inasistencia	El texto se trata como dato literal o se rechaza; no modifica la consulta ni expone datos.
PS-SEG-09	Inicio de sesión exitoso	Cookie de refresh	La cookie tiene HttpOnly, Secure y SameSite=Strict.
PS-SEG-10	Entorno de despliegue	Credenciales de prueba	No existen cuentas de prueba activas con contraseñas conocidas.
PS-SEG-11	Servicio desplegado	Puerto de MariaDB	La base de datos no está expuesta públicamente; solo es accesible desde servicios autorizados.
PS-SEG-12	Respuesta del backend	Cabeceras HTTP	La API incluye cabeceras de seguridad y no filtra trazas o rutas internas ante errores.

Criterios de aceptación o fallo

Son los criterios que serán considerados para dar por completado el plan de pruebas, por ejemplo: porcentaje esperado de cobertura de las pruebas, cierto porcentaje de casos exitosos, cobertura de todos los componentes, porcentaje de defectos corregidos y que todo error vaya acompañado de un mensaje de validación, entre otros. Cuando un criterio de aprobación sea rechazado, se tomará una acción de acuerdo con el criterio de fallo correspondiente. Todo

criterio de aprobación debe estar acompañado por un criterio de fallo, el cual indica la acción que se tomará durante la implementación del plan cuando se ejecuten las pruebas sobre el proyecto.

ID Criterio	Descripción	Aprobación	Fallo
CA-01	Cobertura de controladores backend	Cada controlador backend debe contar con al menos una prueba para una función crítica de negocio, priorizando validaciones, permisos y manejo de errores.	Si no se alcanza esta cobertura mínima, los controladores pendientes se documentarán como deuda técnica y se planificarán para el siguiente sprint.
CA-02	Ejecución de pruebas CI	El 100% de las pruebas automatizadas disponibles debe ejecutarse correctamente en GitHub Actions antes de fusionar un pull request hacia la rama main.	Si una o más pruebas fallan, el pull request permanecerá bloqueado hasta identificar, corregir y validar la causa del fallo.
CA-03	Validación funcional de HU-10	El ministro puede registrar que no asistirá a una actividad, ingresar un motivo y el sistema genera una notificación individual para su coordinador de ministros.	Si el motivo no se guarda, la inasistencia no se registra o el coordinador no recibe la alerta, la funcionalidad no será aceptada hasta corregir el defecto.
CA-04	Regresión de funciones principales	Los flujos principales de autenticación, reservas, asignación de tareas y notificaciones deben ejecutarse sin defectos bloqueantes durante las pruebas integrales.	Si se identifica un defecto bloqueante o pérdida de información en alguno de estos flujos, se suspenderá la aprobación de dicho módulo hasta su corrección.

Criterios de suspensión y reanudación

En esta sección se describen los criterios de suspensión, los cuales establecen claramente bajo qué condiciones se detiene un conjunto de casos de prueba; por ejemplo, cuando existen defectos que impiden ejecutar más casos de prueba, se alcanza cierto porcentaje de casos fallidos o se cumple cualquier otra condición especificada. Los criterios de reanudación establecen bajo qué condiciones se reanudarán las pruebas; por ejemplo, cuando se entregue una nueva versión de prueba o se realicen las configuraciones necesarias. Para cada criterio de suspensión debe existir un criterio de reanudación.

Criterio de suspensión	Criterio de reanudación
Un caso de prueba revela un defecto que bloquea la continuación de las pruebas del módulo afectado.	Las pruebas se reanudan cuando el defecto ha sido corregido, revisado y validado con la ejecución exitosa de los casos relacionados.
El entorno de pruebas no está disponible o presenta fallos de infraestructura, como indisponibilidad de la base de datos, contenedores, dependencias o GitHub Actions.	Las pruebas se reanudan cuando el entorno está disponible, las dependencias necesarias han sido restauradas y se verifica una ejecución básica correcta.
No existen datos de prueba, credenciales o configuraciones válidas para reproducir el escenario de prueba.	Las pruebas se reanudan cuando los datos, credenciales y configuraciones necesarias han sido preparados y el escenario puede reproducirse de forma controlada.
Existe ambigüedad en un requisito o criterio de aceptación que impide determinar el resultado esperado.	Las pruebas se reanudan cuando el equipo o Product Owner aclara el requisito y el caso de prueba se actualiza con el comportamiento esperado.

Infraestructura

Especificación del entorno de desarrollo, lenguajes, frameworks, gestores, herramientas externas, código de sistemas heredados y todo lo necesario para llevar a cabo las pruebas.

El entorno técnico y la infraestructura sobre la cual se ejecutan y validan las pruebas del proyecto están diseñados para garantizar velocidad, aislamiento y una integración continua sin fricciones.

El framework utilizado para las pruebas tanto para el frontend (React) y el backend (Node.js/Express) fue Vitest ya que su velocidad nativa y soporte directo para TypeScript permiten una ejecución ágil. En ambos entornos, las suites de pruebas se disparan de manera estandarizada mediante el comando `npm test` (o `npm run coverage`), facilitando su uso para cualquier desarrollador del equipo. Para garantizar la velocidad y el determinismo de las pruebas del backend no se conectó con la base de datos real del proyecto, en lugar de se utiliza la API de mocks de Vitest (`vi.mock`) para interceptar el pool de conexiones de la base de datos, las respuestas simuladas de la base de datos devuelven objetos tipados estrictamente como `RowDataPacket` (para consultas `SELECT`) y `ResultSetHeader` (para operaciones `INSERT`, `UPDATE`, `DELETE`), emulando con total fidelidad el comportamiento del driver `mysql2`. Para pruebas locales más exhaustivas que requieran levantar todo el ecosistema, se utiliza Docker Compose. El archivo de composición levanta un contenedor aislado con el motor de base de datos MariaDB, asegurando que todos los desarrolladores compartan el mismo esquema y configuración física sin alterar sus máquinas locales. Se ha configurado un flujo CI/CD de trabajo automatizado que actúa como puerta de seguridad. Este workflow se dispara automáticamente al crear o actualizar un Pull Request hacia la rama `main` o `develop`. Obliga a que la suite completa de pruebas pase exitosamente antes de que GitHub permita realizar el Merge del código, asegurando la calidad continua del software.

Suposiciones

Para la correcta ejecución, interpretación y validez de las pruebas definidas en este plan, se dan por sentadas las siguientes condiciones previas y supuestos del entorno:

- **Fidelidad del Entorno de Desarrollo y Pruebas:** Se asume que la configuración local del entorno de desarrollo orquestada con Docker Compose replica fielmente el comportamiento, dependencias y arquitectura del servidor de producción.

- Estabilidad de la Infraestructura y CI/CD: Se asume que los servicios de infraestructura en la nube y los flujos automatizados en GitHub Actions se mantendrán operables, con conectividad estable y sin cambios no documentados durante el periodo de ejecución del sprint.
- Representatividad de los Datos de Prueba: Se da cierto que los scripts de inicialización y semillas de la base de datos contienen un conjunto de datos suficiente, válido y representativo del uso real en la gestión parroquial
- Claridad de Requisitos y Criterios de Aceptación: Se asume que las historias de usuario prioritarias del sprint cuentan con especificaciones y criterios de aceptación definidos sin ambigüedades que permitan verificar objetivamente los resultados esperados.
- Disponibilidad y Perfil del Usuario para Pruebas de Usabilidad: Se asume la disponibilidad oportuna de usuarios finales o perfiles representativos en los rangos de edad objetivo del sistema para efectuar las validaciones de experiencia de usuario y usabilidad de la interfaz.

Riesgos

En esta sección se especificarán los riesgos que pueden afectar directa o indirectamente los resultados de las pruebas. Identificar las acciones preventivas y correctivas de los riesgos definidos permite tomar decisiones rápidas y eficientes, pues el análisis de los riesgos y sus acciones ya se realizó. Es importante documentar los riesgos de manera organizada y comprensible para todas las personas involucradas en el proyecto.

No.	Riesgo	Probabilidad ad (1-5)	Impacto (1-5)	Severidad (Prob. x impacto)	Plan de mitigación
-----	--------	--------------------------	------------------	-----------------------------------	--------------------

No.	Riesgo	Probabilidad (1-5)	Impacto (1-5)	Severidad (Prob. x impacto)	Plan de mitigación
1	Retrasos en la implementación de las funcionalidades.	2	5	10	Evaluar el avance del desarrollo de las funcionalidades y planificar con suficiente tiempo.
2	Los usuarios no están listos para las pruebas de aceptación (UAT).	1	5	5	Coordinar con las oficinas centrales la selección temprana de los usuarios.
3	Los usuarios no están conformes con el producto en las pruebas de aceptación	2	4	8	No entregar el producto tan tarde a los usuarios, dejar una ventana de tiempo suficiente suponiendo que el usuario pedirá correcciones.
4	Las historias de usuario escritas no fueron precisas, y describen funcionalidades mal abordadas o innecesarias.	3	3	9	Revisar constantemente las historias de usuario y compararlas con los requerimientos que muestran los product owners.
5	Cambio en los schemas borran los datos del volumen en docker y el resto del equipo no es informado.	3	2	6	Siempre que se haga un cambio de ese estilo, informar al resto del equipo para que vuelvan a cargar los volúmenes y tengan información de la base de datos actualizada.
6	Cambios nuevos realizados pueden afectar al desarrollo que ya funcionaba anteriormente	4	4	16	Realizar pruebas de regresión en cualquier adición de cambios nuevos en la app.

No.	Riesgo	Probabilidad (1-5)	Impacto (1-5)	Severidad (Prob. x impacto)	Plan de mitigación
7	Rama develop sin protección de testing puede provocar que muchos cambios con errores lleguen a la rama develop, que al intentar mergear a main no pasen los test. El problema es que al estar todo en develop no se sabe de quien es responsabilidad.	4	3	12	Proteger también la rama develop para que corra los test automáticamente y así cualquier merge a develop estará limpio y sin problemas. Si algún test resulta fallido, la persona encargada de esa rama deberá resolverlo.
8	Pruebas de usabilidad insuficientes o realizadas a personas que no representan las edades objetivo de nuestra plataforma pueden provocar una UX/UI difícil de utilizar y de entender para personas mayores.	4	5	20	Prestar especial atención en las pruebas de usabilidad, no infravalorarlas y tomarlas con seriedad. Buscar usuarios que reflejen realmente las edades a las que está orientada la app y realizar suficientes pruebas para obtener resultados significativos.
9	Casos de test que requieran polling pueden dar falsos negativos si no se espera el suficiente tiempo para que se ejecute el poll.	2	3	6	En esos casos específicos de test, como los test de notificación, se deberá especificar un intervalo de al menos 60s antes de verificar el resultado.

Referencia de la plantilla

Crispin, L., & Gregory, J. (2009). *Agile testing: A practical guide for testers and agile teams*.

Pearson Education.

Plantilla de Plan de Pruebas. Universidad Nacional Andrés Bello.